

Employee Timesheet Analytics Project – Leapfrog Tech

Submitted by:

Prajwol Regmi

Project Overview:

Leapfrog Tech gave me two datasets, employee records and timesheet punches, as part of this 2 week assignment. The brief was to take that raw data all the way through to something usable: a clean relational database, a REST API on top of it, business KPIs written in SQL, a few visualizations, and an automated pipeline holding the whole thing together. Nothing about the data was particularly clean going in. The timesheet files overlap each other, some employee IDs on timesheet rows do not exist anywhere in the employee master file, and pay codes are sometimes packed together in a single field.

I broke the work into stages instead of building everything at once. First get the raw files into a proper medallion pipeline (Bronze, Silver, Gold) so the data is trustworthy before anything else touches it. Then write the SQL analytics against the cleaned gold tables. Then build a small local database and API that could serve that same data outside of Databricks. Then wire in authentication, orchestration, and the remaining pieces once the core was working.

Tech stack used across the project:

- Databricks, PySpark and Delta Lake for the Bronze, Silver and Gold layers
- Spark SQL for the KPI queries
- Databricks Workflows for orchestrating the notebooks
- Python, FastAPI and SQLAlchemy for the API
- SQLite as the local database
- JWT (pyjwt) and bcrypt for authentication
- pandas and s3fs for the local data loading step, with MinIO used to test the S3 path
- A Databricks dashboard for the visualizations

Engineering Decisions:

Databricks for the pipeline. This was mostly a comfort call. I already know PySpark and Databricks well, and with timesheet volumes in the hundreds of thousands of rows, Spark was a reasonable fit for the transformation work anyway.

SQLite over Postgres for the API. I did not want to add the overhead of running and configuring a separate database server for what is essentially a local demo of the API layer. SQLite is a single file, needs no setup, and is enough to prove the API works end to end. SQLAlchemy sits between the API and the database specifically so that swapping SQLite for Postgres later is a one line change to the connection string, not a rewrite of the models or the routes, in case this ever needs to run somewhere with concurrent write load.

No foreign key between employees and timesheets. This was deliberate, not something I ran out of time to fix. A number of timesheet rows reference employee IDs that are not in the employee master file. The Gold layer keeps these rows instead of dropping them, because dropping them would understate hours worked and hide a real data quality signal instead of surfacing it. A hard foreign key would reject every one of those rows the moment you tried to load them, so timesheets.client_employee_id is indexed for fast lookups but is not a real foreign key, and a boolean column, employee_master_match, marks which rows do and do not line up with a known employee.

Dropping some Silver columns from the API database. The Silver CSVs carry pipeline bookkeeping fields like `_source_file` and `_ingested_at`, and personal fields like date of birth, home address, phone number and email. None of that is something the API needs to serve, so both categories were left out of the employees and timesheets tables. Everything else, all of the actual business columns, was kept as is.

Keeping pay code as one flat column. Silver produces a proper pay code bridge table and a pay code dimension table for cases where more than one code applies to a single punch. Nothing the API or the required KPIs actually need that breakdown, they need the flags already derived from it (`has_overtime_pay_code`, `has_leave_pay_code`), so I left those two extra tables out of the API database rather than adding joins for a feature nothing was using.

JWT with two hardcoded accounts instead of a real user system. The assignment asks for authentication and authorization, not a full account management system, so I built exactly that: one admin account that can read and write, one viewer account that can only read. Passwords are hashed with `bcrypt` rather than kept in plain text, and tokens are signed with `pyjwt`, both small and well tested libraries rather than anything I wrote myself.

Databricks Workflows instead of Luigi or Airflow. Every notebook in this project depends on Spark and Unity Catalog, so none of them can run outside Databricks. Bringing in a separate orchestration tool locally would have meant writing code just to remotely trigger notebooks over an API and managing credentials for that, when Databricks already does this natively. The job defines explicit task dependencies (bronze ingestion, then silver employee, then silver timesheet, branching into the two gold notebooks, converging at the data quality checks), each task has retries configured, and Databricks' repair run feature lets you rerun just the failed task and whatever depends on it instead of the whole pipeline.

Local files and MinIO both supported, at the loader level. The assignment asks for the option to load from local files or S3 compatible storage. The Databricks notebooks already own the real extract step against a Databricks Volume, and reworking that for S3 would mean touching Spark cluster configuration for no real benefit. The place this actually matters is the local loader that fills the SQLite database, so that is where the source is configurable through an environment variable. I tested this against a real local MinIO server rather than leaving it as untested code.

Challenges along the way. Getting the API to talk to the database and recognize a login correctly took some trial and error, mainly understanding how a bearer token needed to flow from the login response into the Authorization header on every following request. Getting the punch in and punch out columns to parse correctly into proper date and timestamp types in Spark also took much longer than expected, one malformed row was enough to throw off type inference for the whole batch, which meant going back to how Bronze was structured and adding a quarantine step that pulls out malformed rows before they reach Silver, tagging each one with a reason, so the rest of the pipeline runs cleanly. Separately, once the Silver CSVs were ready to commit, GitHub rejected the push since the timesheet exports were over its 100MB per file limit, which meant resetting a couple of commits and compressing the offending files before committing again properly.

Setup Instructions:

You need Python 3.11 or newer, and the CSV exports, already included in the repository under data/bronze and data/silver, with the large timesheet files stored as gzip.

```
git clone <repository url>
cd employee_timesheet_analysis/api
```

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

That installs FastAPI, SQLAlchemy, pandas, the JWT and password hashing libraries, and s3fs for the optional S3 loading path.

Usage Guide:

Building the local database

```
python -m db.load_data
```

This creates api/app.db if it does not exist, wipes both tables, and reloads them from data/silver/employee.csv and data/silver/timesheet.csv.gz. Safe to run again any time the source data changes.

To load from MinIO or S3 instead, set these before running the same command:

```
export DATA_SOURCE=s3
export S3_BUCKET=your-bucket
export S3_PREFIX=silver
export S3_ENDPOINT_URL=http://localhost:9000
export S3_ACCESS_KEY=your-key
export S3_SECRET_KEY=your-secret
```

Running the API

```
uvicorn main:app --reload
```

Open <http://127.0.0.1:8000/docs> for the interactive Swagger page, or <http://127.0.0.1:8000/redoc> for a read only reference view.

Log in first through POST /login with one of two demo accounts:

- admin / admin123, full read and write access
- viewer / viewer123, read only

Copy the access_token from the response, click Authorize on the docs page, and paste it in. Every endpoint besides /health and /login requires this.

Available endpoints:

- POST /employees, GET /employees, GET /employees/{employee_id}, PUT /employees/{employee_id}, DELETE /employees/{employee_id} (write operations need the admin role)
- GET /timesheets, filterable by employee id, start date and end date, paginated with limit and offset
- GET /timesheets/{employee_id}, the same date filters, scoped to one employee

Running the ETL pipeline and generating reports:

The Databricks Workflow named employee_timesheet_etl runs all six notebooks in order: bronze ingestion, then silver employee, then silver timesheet, branching into the two Gold notebooks (workforce analytics and attendance analytics), converging at the data quality checks, with the SQL analytics task running alongside. Trigger it manually from Jobs and Pipelines in the Databricks workspace, or add a schedule from the same screen for it to run unattended.

The SQL KPI queries live in notebooks/07_sql_analytics.sql and run against the Gold tables once the pipeline has completed. The dashboard, with rolling average working hours, an attendance exception breakdown, and a Genie chat panel for ad hoc questions, is published in the Databricks workspace under Dashboards.

Schema Documentation:

Two tables, employees and timesheets, both living in a single SQLite database file, created and populated by the loader script.

employees	timesheets
client_employee_id PK ->	client_employee_id
first_name ->	timesheet_id PK
last_name ->	punch_apply_date
department_name ->	punch_in_datetime
job_title ->	punch_out_datetime
hire_date ->	pay_code

term_date ->	is_late_arrival
active_status ->	is_early_departure
tenure_days ->	is_overtime
is_early_attrition ->	employee_master_match

employees table, primary key client_employee_id, 28 columns in total: identity fields (first name, last name, preferred name, full name), job fields (job code, job title, job start date, department id, department code, department name, organization name), manager fields (manager employee id, manager exists), employment fields (fte status, scheduled weekly hours, hire date, recent hire date, anniversary date, term date, active status, employment status, termination reason), and tenure fields already computed upstream (tenure days, tenure years, tenure end date, days to termination, is early attrition).

timesheets table, primary key timesheet_id (a hash generated from the natural key of employee, punch date, and punch in and out times), 38 columns in total: identifiers and timing (client employee id, employee master match, punch apply date, punch in and out timestamps, scheduled start and end timestamps), department fields, pay code fields (pay code, pay code count, has multiple pay codes, has overtime pay code, has leave pay code, punch comments), duration fields (hours worked, actual and scheduled duration in minutes and hours, variance fields), and the attendance flags computed with a five minute grace period (late arrival, early departure, overtime, excessive duration).

client_employee_id and punch_apply_date are indexed on the timesheets table, since those are exactly the two filters the API exposes. The relationship between the two tables is intentionally not enforced as a foreign key, for the reason explained in the Engineering Decisions section above.

THANK YOU